Network Traffic Analysis Using Wireshark

Category: Network Security / Packet Analysis
Tool: Wireshark
Protocols Analyzed: DNS, TCP, HTTP
Devarth Patel

1. Overview

In this lab, I focused on analyzing captured network traffic using Wireshark, an open-source network protocol analyzer. The objective was to examine how a client communicates with a web server by analyzing DNS queries, TCP connections, and HTTP requests.

Using a packet capture, I investigated the communication between a client and a web server to identify the requested domain, destination IP address, HTTP requests, and web server information.

This gave me some practical experience interpreting network packets and demonstrated how network traffic analysis can support cybersecurity investigations and network monitoring.

2. Objectives

The primary objectives of this lab were to:

- Analyze captured network traffic using Wireshark.
- Identify a domain name queried through DNS.
- Determine the IP address of the contacted web server.
- Examine HTTP GET requests and identify the requested resources.
- Determine the number of HTTP requests sent by the client.
- Identify the web server software and operating system reported in the HTTP response.
- Understand how packet analysis contributes to network security investigations.

3. Tools and Environment

Component	**Description**
Network Analysis Tool	Wireshark
Traffic Source	Provided packet capture
Protocols Analyzed	DNS, TCP, HTTP
Target Web Server	gaia.cs.umass.edu
Analysis Method	Offline packet inspection

4. Methodology

4.1 Examining Network Traffic

I began by reviewing the packet capture in Wireshark.

The packet list displayed network communications that involve multiple protocols, including DNS, TCP, and HTTP. I checked the source and destination addresses, protocol classifications, and information fields to identify relevant network activity.

The captured traffic showed the process of resolving a hostname, establishing TCP communication, and requesting web resources using HTTP.

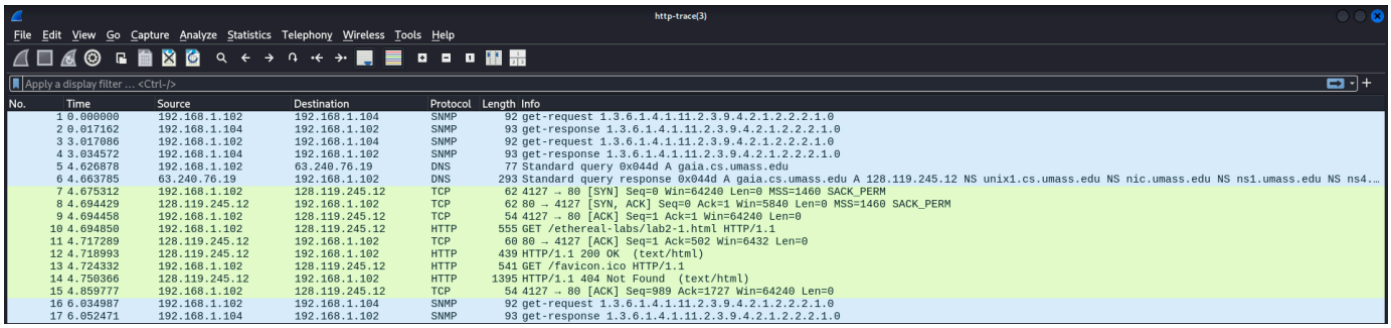


Figure 1: Wireshark packet capture showing DNS resolution, TCP communication, and HTTP requests between a client and web server.

4.2 DNS Query Analysis

I examined the DNS packets to find the domain name requested by the client.

DNS translates human-readable domain names into IP addresses, ad allows systems to locate and communicate with remote servers.

The DNS query in packet No. 5 showed the following hostname:

> Queried Hostname: gaia.cs.umass.edu

This showed how DNS requests can be inspected to find domains that a client attempts to access.

4.3 Identifying the Web Server IP Address

After getting the requested hostname, I examined the network traffic to get the IP address of the contacted web server.

> Web Server IP: 128.119.245.12

The examination of network-layer addresses helps analysts find endpoints involved in communication.

4.4 HTTP Request Analysis

Next, I examined the HTTP packets to find the resources requested by the client.

The first HTTP GET request appeared in packet No. 10. The requested URL was:

> http://gaia.cs.umass.edu/ethereal-labs/lab2-1.html

The request targeted the following resource:

> /ethereal-labs/lab2-1.html

The HTTP GET request tells a web server to retrieve a specified resource.

4.5 Counting HTTP Requests

I reviewed the HTTP traffic to get the total number of requests sent from the client to the server.

The packet capture had **two HTTP GET requests**.

Packet Number	**HTTP Request**	**Purpose**
No. 10	GET /ethereal-labs/lab2-1.html	Request for the main web page
No. 13	GET /favicon.ico	Request for the website favicon

This showed that accessing a web page can generate extra requests for resources that are associated with that page.

4.6 Identifying Web Server Software

Finally, I looked at the HTTP response details to identify information about the software running on the contacted web server.

The HTTP response had the following server identification information:

> **Server:** Apache/2.0.40 (Red Hat Linux)

This meant that the responding server identified itself as Apache version 2.0.40 running on Red Hat Linux.

HTTP response headers can provide useful information about a web server's software environment. But, server headers can also be modified or suppressed, so they can’t be treated as proof of the underlying configuration.

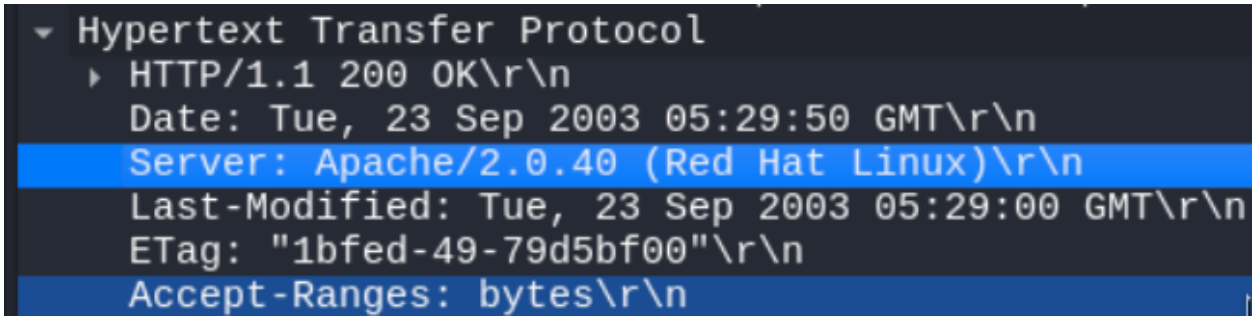


Figure 2: Wireshark HTTP response details displaying the server identification header, Apache/2.0.40 (Red Hat Linux).

5. Results and Findings

Analysis Item	**Result**
DNS Hostname	gaia.cs.umass.edu
Web Server IP Address	128.119.245.12
First HTTP Request	GET /ethereal-labs/lab2-1.html
Full Requested URL	http://gaia.cs.umass.edu/ethereal-labs/lab2-1.html
Total HTTP GET Requests	2
Server Software	Apache/2.0.40
Reported Operating System	Red Hat Linux

This analysis demonstrated how DNS, TCP, and HTTP work together during client-server communication. DNS traffic helped identify the requested hostname, TCP traffic showed transport-layer communication between endpoints, and HTTP traffic revealed the requested web resources and server identification information.

6. Cybersecurity Relevance

This lab helped practice several practical applications:

- **Network Visibility:** Examining captured packets gives insight into how systems communicate and which protocols are used.
- **Incident Investigation:** DNS queries, destination IP addresses, and HTTP requests can help analysts reconstruct network events and investigate potentially suspicious activity.

- **Protocol Analysis:** Understanding how DNS, TCP, and HTTP interact provides a foundation for identifying unusual network behavior.
- **Information Exposure:** Unencrypted HTTP communication can show requested resources and other application-layer information.
- **Server Fingerprinting:** HTTP response headers may disclose software information relevant to security assessments and vulnerability investigations.

8. Conclusion

This lab provided hands-on experience analyzing network traffic using Wireshark. By examining DNS, TCP, and HTTP packets, I identified the queried hostname, destination web server IP address, requested URLs, number of HTTP requests, and server software information.